

Report conversation

こちらのコードを、シンプルながら読みやすいコードにリファクタリングしてください。

```
let lastCommandTime: number | null = null;
let lastSongCommandTime: number | null = null;
if (state.undoCommands.length !== 0) {
  lastCommandTime =
    state.undoCommands[state.undoCommands.length
- 1].unixMillisec;
}
if (state.undoSongCommands.length !== 0) {
  lastSongCommandTime =

state.undoSongCommands[state.undoSongCommands.le
ngth - 1]
  .unixMillisec;
}
if (lastCommandTime !== null &&
lastSongCommandTime !== null) {
  return Math.max(lastCommandTime,
lastSongCommandTime);
} else if (lastCommandTime !== null) {
  return lastCommandTime;
} else if (lastSongCommandTime !== null) {
  return lastSongCommandTime;
}
return null;
```

以下のコードは、指定されたコードをよりシンプルかつ読みやすくリファクタリングしたものです。ここでは、繰り返しを避け、コードの意図を明確にしています。

typescript

 Copy code

```
function getLastCommandTime(state: any): number | null {
  const getLastTime = (commands: any[]): number | null => {
    const lastCommand = commands[commands.length - 1];
```

```
    return lastCommand ? lastCommand.unixMillisec : null;
  };

  const lastCommandTime = getLastTime(state.undoCommands);
  const lastSongCommandTime = getLastTime(state.undoSongCommands);

  return lastCommandTime && lastSongCommandTime
    ? Math.max(lastCommandTime, lastSongCommandTime)
    : lastCommandTime || lastSongCommandTime;
}
```

このリファクタリングにより、次の改善が行われています：

1. **関数の導入**：コードの重複を避けるために、共通のロジックを扱う `getLastTime` 関数を導入しました。これにより、`undoCommands` と `undoSongCommands` の最後のコマンド時間を取得する処理を簡素化しています。
2. **条件式の簡素化**：最後のコマンド時間と最後の曲コマンド時間の比較に三項演算子を使用し、コードの可読性を高めています。また、`||` 演算子を用いて、どちらかの値が `null` ではない場合にその値を返すようにしています。

このリファクタリングにより、コードはより簡潔で読みやすくなり、同じ機能を保持しながらメンテナンスしやすくなります。